

Factoring algorithm

July 13, 2025

Shor's algorithm for integer factorization

Shor's algorithm, developed by Peter Shor in 1994, is a quantum algorithm that efficiently factorizes large integers, threatening classical encryption like RSA. It exploits quantum mechanics to solve the problem exponentially faster than classical methods.

Supposing the Input is an odd, composite integer N that is not a prime power. The algorithm consists on iterating the following steps:

1. Randomly choose $x \in \{2, \dots, N - 1\}$.
2. Compute $d = \gcd(x, N)$.
3. If $d > 1$, then output $b = d$ and $c = N/d$ and stop. Otherwise, continue to the next step knowing that $x \in \mathbb{Z}_N^*$.
4. Let r be the order of $x \bmod N$, found using order-finding.
5. If r is even and $x^{r/2} \not\equiv -1 \pmod{N}$:
 - (a) Compute $a = x^{r/2} - 1 \pmod{N}$.
 - (b) Compute $d = \gcd(a, N)$.
 - (c) If $d > 1$, then output $b = d$ and $c = N/d$ and stop.
6. If this point is reached, the iteration has failed to find a factor of N .

Most of these steps can be done efficiently using classical methods. However no known classical algorithm can perform order-finding efficiently. On the following section, we follow the steps made to factor $N = 35$ using a quantum computer.

Factoring $N = 35$

We start by choosing a random number x that doesn't have any common factor with $N = 35$. For example, $x = 17$.

Phase estimation

Having chosen x , let's discuss the quantum state we work with. We can divide the state in two different registers. We call the first register the estimation register, composed of t qubits, and the second register the target register.

We will perform the quantum phase estimation (QPE) algorithm shown in Figure 1.

The QPE algorithm consists on preparing the estimation register in a superposition and the target register in a certain state $|\psi\rangle$, then applying controlled- U^{2^j} gates ($j = 0, 1, \dots, t - 1$) on the target state, and finally applying the inverse Quantum Fourier Transform to the estimation register.

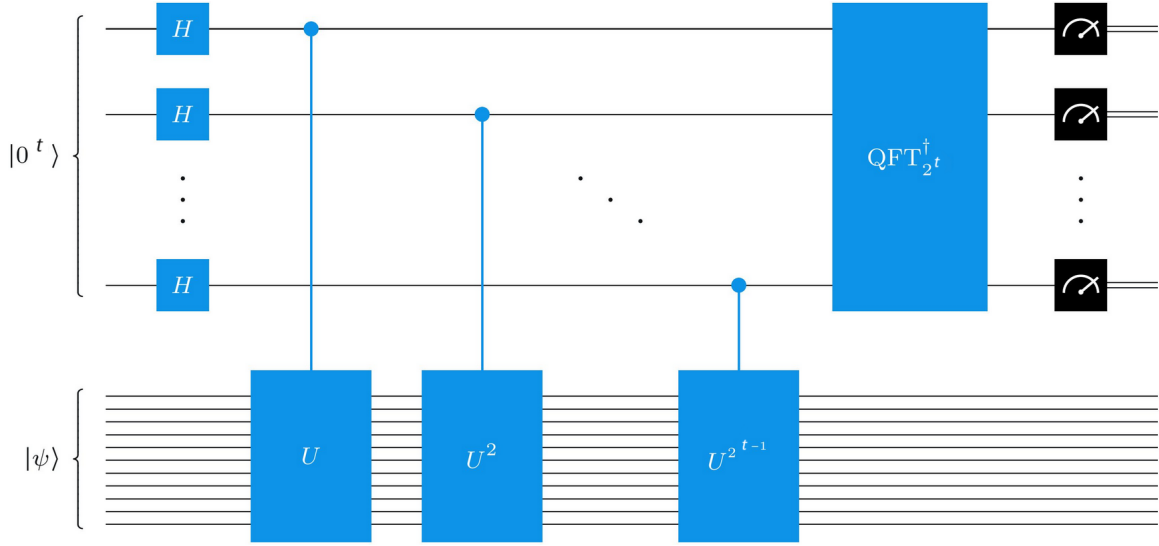


Figure 1: Quantum phase estimation circuit [1].

This algorithm determines the phase θ of an eigenstate $|\psi\rangle$ of a unitary operator U , where $U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$. In our case we are interested in the following unitary:

$$U_{N,x}|y\rangle = |xy \bmod N\rangle \quad (\text{for each } y \in \mathbb{Z}_N).$$

The phase of the eigenvalues of this unitary is closely related to the order of x modulo N , r . Note that r applications of U on state $|1\rangle$, return the state to $|1\rangle$. For $N = 35$ and $x = 17$:

$$\begin{aligned} U_{35,7}|1\rangle &= |17\rangle \\ U_{35,7}^2|1\rangle &= |9\rangle \\ &\vdots \\ U_{35,7}^{r-1}|1\rangle &= |33\rangle \\ U_{35,7}^r|1\rangle &= |1\rangle. \end{aligned}$$

This behavior is plotted in Figure 2, where we can easily see that the period or order $r = 12$. But how can we obtain r from the phase θ ? We need to choose a suitable target state $|\psi\rangle$.

Consider the state $|u_0\rangle$:

$$\begin{aligned} |u_0\rangle &= \frac{1}{\sqrt{r}}(|1\rangle + |17\rangle + \dots + |33\rangle) \\ U_{35,7}|u_0\rangle &= \frac{1}{\sqrt{r}}(|17\rangle + |9\rangle + \dots + |1\rangle) \\ &= |u_0\rangle. \end{aligned}$$

$|u_0\rangle$ is an eigenstate of $U_{35,7}$ with eigenvalue 1, which is not very useful. However, we can define a similar eigenstate in which the phase of each $|k\rangle = |x^k \bmod N\rangle$ state is proportional to k in the following way:

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i \frac{sk}{r}} |x^k \bmod N\rangle.$$

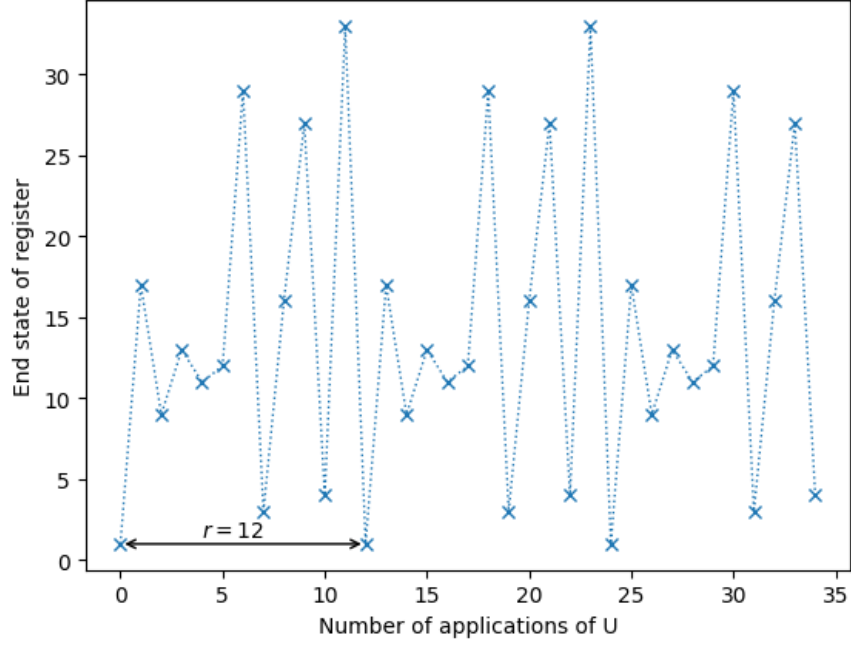


Figure 2: Effect of successive applications of $U_{N,x}$ on the target register.

$|u_s\rangle$ is an eigenstate of $U_{N,x}$ with eigenvalue $e^{2\pi i \frac{s}{r}}$:

$$\begin{aligned}
 |u_s\rangle &= \frac{1}{\sqrt{r}}(|1\rangle + e^{-2\pi i \frac{s}{r}} |17\rangle + \dots + e^{-2\pi i \frac{s(r-1)}{r}} |33\rangle) \\
 U_{35,7} |u_s\rangle &= \frac{1}{\sqrt{r}}(|17\rangle + e^{-2\pi i \frac{s}{r}} |9\rangle + \dots + e^{-2\pi i \frac{s(r-1)}{r}} |1\rangle) \\
 &= \frac{1}{\sqrt{r}} e^{2\pi i \frac{s}{r}} (|1\rangle + e^{-2\pi i \frac{s}{r}} |17\rangle + \dots + e^{-2\pi i \frac{s(r-1)}{r}} |33\rangle) \\
 &= e^{2\pi i \frac{s}{r}} |u_s\rangle.
 \end{aligned}$$

The phase $\theta = \frac{s}{r}$, which depends on r . We now have a unique eigenstate for each integer value of s , where $0 \leq s \leq r-1$. We cannot prepare the target register in one of these states, but there is one last trick we can use to obtain a state $|\psi\rangle$ from which we can infer r . If all $|u_s\rangle$ are summed:

$$\begin{aligned}
 \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} &= \frac{1}{r} (|1\rangle + |17\rangle + |9\rangle + \dots \\
 &\quad |1\rangle + e^{-\frac{2\pi i}{r}} |17\rangle + e^{-\frac{4\pi i}{r}} |9\rangle + \dots \\
 &\quad |1\rangle + e^{-\frac{4\pi i}{r}} |17\rangle + e^{-\frac{8\pi i}{r}} |9\rangle + \dots \\
 &\quad \vdots \\
 &\quad |1\rangle + e^{-\frac{2(r-1)\pi i}{r}} |17\rangle + e^{-\frac{4(r-1)\pi i}{r}} |9\rangle + \dots) = |1\rangle \quad \left(\text{since } \sum_{s=0}^{r-1} e^{2\pi i \frac{s}{r}} = 0 \right).
 \end{aligned}$$

Therefore, the computational basis state $|1\rangle$ turns out to be a superposition of all $|u_s\rangle$, which means

if we do QPE on $U_{N,x}$ using the state $|1\rangle$, we will measure a phase:

$$\theta = \frac{s}{r}.$$

In reality, QPE returns an estimation of θ given by:

$$\theta \approx \frac{y}{2^t},$$

where $y \in 0, \dots, 2^{t-1}$. So the quality of the approximation depends on the number of qubits used for the estimation register, t .

Order finding

Now that we have an algorithm to estimate $\frac{y}{2^t} \approx \frac{s}{r}$ with a random $s \in \{0, \dots, r-1\}$, we can obtain the fraction $\frac{u}{v}$ satisfying $u \in \{0, \dots, N\}$ and $v \neq 0$, that is closest to $\frac{y}{2^t}$. This can be efficiently done classically, using the continued fraction algorithm.

Therefore, only one question remains. How much precision do we need to correctly determine $\frac{u}{v} = \frac{s}{r}$? We can get a sense of the answer if we ask ourself the simpler question of how much larger must t be in order to distinguish $\frac{1}{r}$ from the close number $\frac{1}{r+1}$. If we want to make sure that we don't mistake $\frac{1}{r}$ for $\frac{1}{r+1}$, it suffices to use enough precision to guarantee that a best approximation $\frac{y}{2^t}$ is closer to $\frac{1}{r}$ than it is to $\frac{1}{r+1}$:

$$\left| \frac{y}{2^t} - \frac{1}{r} \right| < \frac{1}{2} \left(\frac{1}{r} - \frac{1}{r+1} \right) = \frac{1}{2r(r+1)}.$$

Since we don't know r , but $r < N$, we can use the N to ensure that we use enough precision:

$$\left| \frac{y}{2^t} - \frac{1}{r} \right| \leq \frac{1}{2N^2},$$

from which we can take $t = 2\log(N) + 1$ to ensure that we have a high chance to obtain a good estimation.

There is some other inconveniences when trying to determine $\frac{u}{v} = \frac{s}{r}$. For example, it might happen that $s = 0$, and we don't get any information about r . But that's the only value of s where that happens. Another problem is that we might be unlucky, and when s is nonzero, it has common factors with r . Then v obtained from the continued fraction algorithm is not r , but a factor of r . It turns out that this isn't a huge deal if we perform many measurements with s chosen uniformly at random. Then we're very likely to recover r after just a few samples, if our guess for r is the least common multiple of all the values for v that we observe.

Let's put these ideas to the test. With $N = 35$, we should use at least $t = 2 \log(35) + 1 = 11.26$, thus $t \geq 12$ for a good chance of success. We start by observing what happens when we try to use a much smaller sized estimation register, with $t = 4$. The QPE circuit and outcome probability distribution are shown in Figure 3. All 16 outcomes have fairly large probability of occurrence, but we observe that from the estimated phases of every one of them, shown in Table 1, no correct answer of $r = 12$ is obtained. The order finding algorithm fails with very low t .

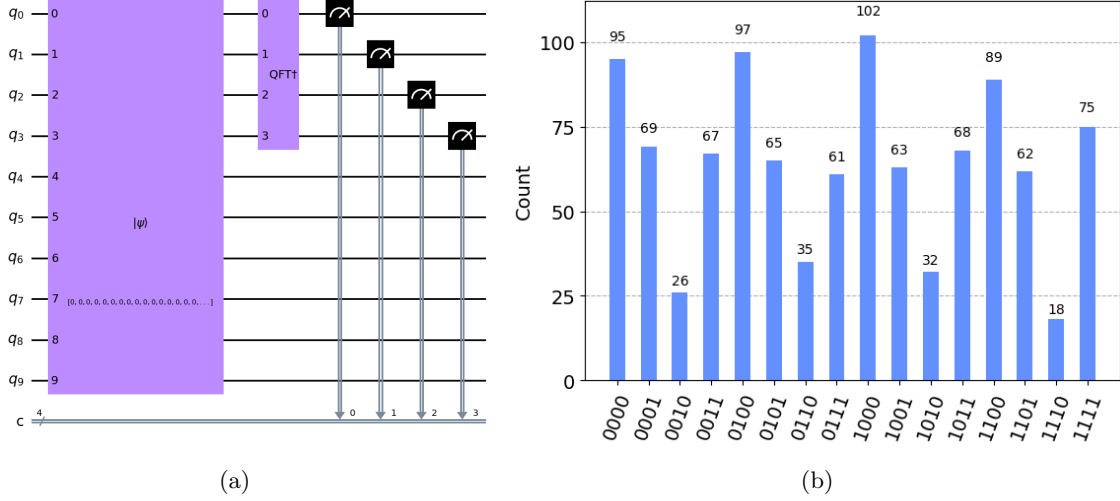


Figure 3: (a) QPE circuit diagram with $t = 4$ and (b) simulated outcome probability distribution, with $N = 35$ and $x = 17$.

Phase	Fraction	Guess for r
$0/16 = 0.00$	$0/1$	1
$8/16 = 0.50$	$1/2$	2
$4/16 = 0.25$	$1/4$	4
$7/16 = 0.44$	$7/16$	16
$12/16 = 0.75$	$3/4$	4
$13/16 = 0.81$	$13/16$	16
$1/16 = 0.06$	$1/16$	16
$9/16 = 0.56$	$9/16$	16
$5/16 = 0.31$	$5/16$	16
$11/16 = 0.69$	$11/16$	16
$3/16 = 0.19$	$3/16$	16
$15/16 = 0.94$	$15/16$	16
$10/16 = 0.62$	$5/8$	8
$6/16 = 0.38$	$3/8$	8
$14/16 = 0.88$	$7/8$	8
$2/16 = 0.12$	$1/8$	8

Table 1: Table of phase fractions and guesses for r

We perform the same simulation for $t = 12$. In this case we have 4096 possible outcomes, but the simulation reveals that around 12 of them are way more probable than the rest. If we focus on the 12 most likely output states, we notice that four of them guess the correct $r = 12$, and the rest of them are factors of 12 (with just 3 odd numbers). Therefore, we conclude that the order finding algorithm would most likely work with a few iterations, thanks to the improved precision we got with a larger t .

The factoring algorithm would end shortly after, by calculating $a_{\pm} = 17^{12/2} \pm 1 \pmod{N}$ and the two factors: $\gcd(a_{\pm}, N) = 5_+, 7_-$.

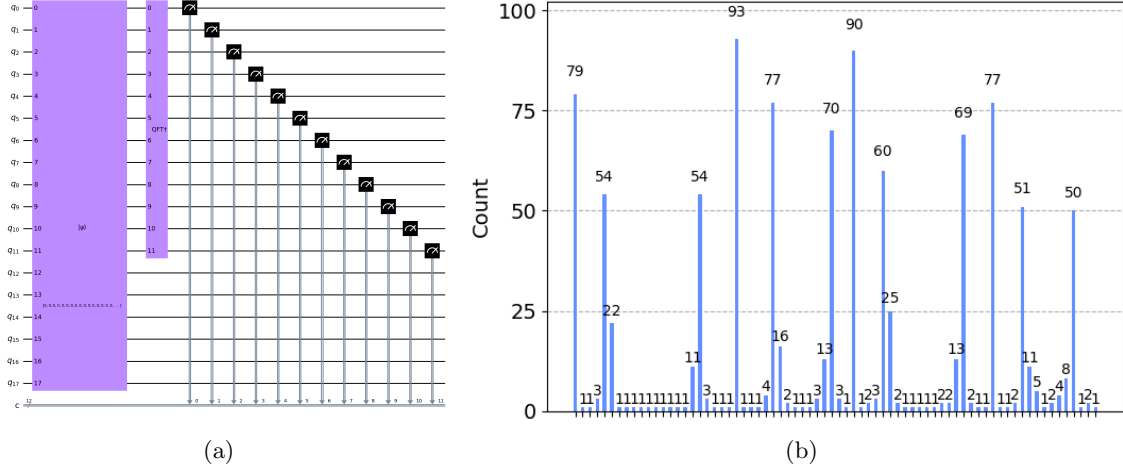


Figure 4: (a) QPE circuit diagram with $t = 12$ and (b) simulated outcome probability distribution, with $N = 35$ and $x = 17$.

Phase	Fraction	Guess for r
$3072/4096 = 0.75$	$3/4$	4
$1024/4096 = 0.25$	$1/4$	4
$2048/4096 = 0.50$	$1/2$	2
$0/4096 = 0.00$	$0/1$	1
$683/4096 = 0.17$	$1/6$	6
$1365/4096 = 0.33$	$1/3$	3
$1707/4096 = 0.42$	$5/12$	12
$341/4096 = 0.08$	$1/12$	12
$2731/4096 = 0.67$	$2/3$	3
$2389/4096 = 0.58$	$7/12$	12
$3755/4096 = 0.92$	$11/12$	12
$3413/4096 = 0.83$	$5/6$	6

Table 2: Table of phase fractions and guesses for r

Different choice of x

We have worked with $x = 17$, which happens to have an order $r = 12$, which is the largest possible for $N = 35$. In reality, x will be chosen at random, and we can end up with $x = 6$ for example. What difference can this make? It turns out that 16 has the lowest possible even order $r = 2$, since $6^2 = 36 = 1 \pmod{35}$. For this case, a t equal to 1 is enough to have a 50% chance of correctly

obtaining r in a single shot, and it remains that for higher t -s. The same can be said for $x = 8$, which has order $r = 4$, when $t > 2$.

Other choices, like $x = 11$ and $x = 16$ lead to an odd order $r = 3$, although in this case these two are the only choices with odd r . If too many choices of x led to an odd order, the algorithm might not be useful. Fortunately, using basic number theory it can be proved that, for a random choice of x , with probability of at least $1/2$ r will be even.

Computational cost

Lastly, we discuss the cost of the performed operations. Let's start by focusing on the Quantum Phase Estimation circuit.

The cost of the t Hadamard gates is $\mathcal{O}(t)$, and the cost of the QFT is $\mathcal{O}(t^2)$. The last pieces are the $U_{N,x}^k$ unitaries, with $k = 1, 2, \dots, 2^{t-1}$. Since k grows exponentially with the number of qubits t , one might think that an exponential number of $U_{N,x}$ are needed to implement them. However, each $U_{N,x}^k$ can be implemented by computing $b = xk \bmod N$ classically, and using the circuit for $U_{N,b}$. This means that the cost of applying t $U_{N,x}$ gates, which have a cost of $\mathcal{O}(t^2)$ each, is $\mathcal{O}(t^3)$. We recall that $t = 2 \log(N) + 1$, meaning that the QPE circuit runs in time $\mathcal{O}(\log(N)^3)$.

In addition to the quantum circuit itself, there are a few classical computations that need to be performed along the way. This includes computing the powers x^k in $\mathbb{Z}_N$ for $k = 2, 4, 8, \dots, 2^{t-1}$, which are needed to create the controlled-unitary gates, as well as the continued fraction algorithm that converts approximations of θ into fractions. In both cases, these computations can be performed by Boolean circuits having cost $\mathcal{O}(\log(N)^3)$.

References

- [1] IBM Quantum Learning. *Fundamentals of Quantum Algorithms: Phase Estimation and Factoring*.
<https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/phase-estimation-and-factoring#factoring-by-order-finding>
 Accessed: February 3, 2025.
- [2] Qiskit contributors. *Implementing Shor's Algorithm in Qiskit*.
<https://github.com/Qiskit/textbook/blob/main/notebooks/ch-algorithms/shor.ipynb>
 Accessed: February 3, 2025.
- [3] quantummasterbarcelonacode. *Quantum Computation - Factoring Code Repository*.
<https://github.com/quantummasterbarcelonacode/QuantumComputation/tree/main/Factoring>
 Accessed: February 3, 2025.